Guia Completo de Estudo e Defesa: COMINDIR04

Deteção de Falhas por Heartbeat e Envio em Lote

Preparação para Defesa Oral

5 de novembro de 2026

Conteúdo

1 O que há de novo em relação à COMINDIR03 **2**

2 Comandos de Compilação e Execução **2**
2.1 Compilação 2
2.2 Execução (Pela ordem correta) 2

3 Análise Detalhada das Alterações e Trechos de Código **3**
3.1 Propriedades de Sistema (`Wire.java`) 3
3.2 Empacotamento e Validação de Lotes (`Batch.java`) 4
3.2.1 Método `encode()` 4
3.2.2 Método `decode()` 4
3.3 O Detetor de Falhas (`HeartbeatMonitor.java`) 5
3.3.1 Método `onHeartbeat()` 5
3.3.2 Método `check()` 6
3.4 Recuperação Assíncrona na Réplica (`PartitionReplica.java`) 6
3.4.1 O gatilho no método `receber()` 6
3.4.2 O método `executarRecuperacao()` 7
3.5 Relógios Lógicos no Processador (`AlarmProcessor.java`) 8

4 Perguntas Fundamentais do Agente e Respostas **9**

5 Simulado de Defesa (Perguntas e Respostas) **10**

6 Cenários de Falha (Perguntas de Defesa) **11**

1 O que há de novo em relação à COMINDIR03

Na COMINDIR03, o líder publicava um `REG` por cada leitura e ficava em silêncio quando não havia trabalho. Isso criava dois problemas graves: o **silêncio ambíguo** (não se sabia se o líder morreu ou se não havia leituras) e a **perda do último registo** (se o líder falhasse antes de publicar, ninguém dava por isso, pois a réplica só detetava lacunas quando recebia um registo com `seq` superior ao esperado).

A COMINDIR04 resolve estas limitações com as seguintes alterações:

- **Envio em Lote (`LOTE`):** O líder acumula as leituras numa lista e publica-as todas de uma vez a cada 5 segundos (Δ). Isto reduz o tráfego, o *overhead* de rede (cabeçalhos TCP/IPFS) e remove carga do IPFS.
- **Heartbeat:** O `LOTE` é enviado mesmo quando está vazio (`n=0`). O lote vazio serve como sinal de vida contínuo, permitindo distinguir "líder sem trabalho"de "líder falhado".
- **Detetor de Falhas (`HeartbeatMonitor`):** Réplicas, produtor e processador monitorizam o silêncio. Se passarem 15 segundos (TIMEOUT = $3 \times \Delta$) sem receber lotes, o líder passa a `SUSPEITO`.
- **Campo `ultimoSeq`:** O cabeçalho do lote indica o último `seq` atribuído. Permite detetar lacunas mesmo que o lote venha vazio (ex: se o líder morreu após gravar o `seq 3` mas antes de o enviar).
- **Recuperação Assíncrona:** Na réplica, a recuperação de lacunas por TCP ocorre numa *thread* separada, para não bloquear a leitura dos heartbeats do IPFS e evitar falsas suspeitas.
- **Incarnação (`inc`):** Identifica univocamente o arranque do líder. Se mudar, é porque o líder reiniciou.

2 Comandos de Compilação e Execução

A ordem de arranque é crucial para o funcionamento do sistema.

2.1 Compilação

Em cada uma das pastas dos projetos (`rep04-lider`, `rep04-replica`, `rep04-produtor`, `rep04-processador`):

```
mkdir -p out
javac -d out src/rep04/*.java

```

2.2 Execução (Pela ordem correta)

1. IPFS Daemon (num terminal separado, deixar a correr)

```
ipfs daemon

```

2. **Réplicas** (em terminais separados)

```
# Replica 0 (guarda p0, p2)
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.PartitionReplica 0 3 0,2 5001

# Replica 1 (guarda p1)
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.PartitionReplica 1 3 1 5001
```

3. **Líderes** (em terminais separados, para k=0, 1, 2)

```
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.PartitionLeader 0 3 127.0.0.1
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.PartitionLeader 1 3 127.0.0.1
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.PartitionLeader 2 3 127.0.0.1
```

4. **Processador de Alarmes** (num terminal)

```
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.AlarmProcessor 3 5001
```

5. **Produtor** (num terminal)

```
java -Drep04.periodo=5000 -Drep04.fator=3 -cp out rep04.Producer 3 5001
```

6. **Análise (após os ensaios)**

```
java -cp rep04-replica/out rep04.HeartbeatCheck 3 <pasta_ensaio> <pasta_ensaio>/falhas.txt
java -cp rep04-replica/out rep04.CausalCheck 3 <pasta_ensaio>
```

3 Análise Detalhada das Alterações e Trechos de Código

3.1 Propriedades de Sistema (`Wire.java`)

```
public static final long PERIODO_HB_MS = Long.getLong("rep04.periodo", 5000L);
public static final int FATOR_TIMEOUT = Integer.getInteger("rep04.fator", 3);
public static final long VERIFICACAO_MS = 1000L;
```

Explicação detalhada: O método `getLong` procura a propriedade de sistema `-Drep04.periodo` passada na linha de comandos. Se não encontrar, usa `5000` como valor padrão. Isto permite configurar o período de heartbeat sem recompilar o código. O `FATOR_TIMEOUT` define o multiplicador para calcular o TIMEOUT. Com `fator=3`, o timeout é $5000 \times 3 = 15000$ ms. O `VERIFICACAO_MS` define o intervalo (1000 ms) com que a thread de verificação acorda para testar o `check()` do monitor. Não é configurável por propriedade de sistema porque a ficha fixa este valor.

3.2 Empacotamento e Validação de Lotes (`Batch.java`)

3.2.1 Método `encode()`

```
    public String encode() {
      StringBuilder sb = new StringBuilder();
      // Cabecalho: LOTE;<endereco>;<porto>;<inc>;<hb>;<ultimoSeq>;<n>
      sb.append("LOTE;").append(address).append(";").append(port).append(";")
        .append(inc).append(";").append(hb).append(";")
        .append(lastSeq).append(";").append(records.size()).append("\n");

      for (SensorRecord r : records) {
        sb.append(r.toLine()).append("\n");
      }
      return sb.toString();
    }

```

Explicação detalhada: Este método é chamado pelo líder. Ele constrói a string do lote linha a linha. A primeira linha é o cabeçalho: `LOTE;endereço;porto;inc;hb;ultimoSeq;n`. O campo `n` é obtido através de `records.size()` (o número de registos no lote). Depois, para cada registo na lista `records`, é chamado o método `r.toLine()` que devolve a string do registo (ex: `1;S01;21.0;...`) e acrescenta-se um `\n` (mudança de linha). Isto empacota todas as leituras numa única mensagem, reduzindo o overhead de rede.

3.2.2 Método `decode()`

```
    public static Batch decode(String msg, int nParticoes) {
      String[] lines = msg.split("\n");
      if (lines.length < 1) throw new IllegalArgumentException("Mensagem vazia");

      String[] header = lines[0].split(";");
      if (header.length != 7 || !header[0].equals("LOTE")) {
        throw new IllegalArgumentException("Cabecalho LOTE malformado");
      }

      String address = header[1];
      int port = Integer.parseInt(header[2]);
      long inc = Long.parseLong(header[3]);
      long hb = Long.parseLong(header[4]);
      long lastSeq = Long.parseLong(header[5]);
      int nHeader = Integer.parseInt(header[6]);

      int numLinhasRegistos = lines.length - 1;
      if (nHeader != numLinhasRegistos) {
        throw new IllegalArgumentException("Inconsistencia: n declarado (" + nHeader + ") != linhas de registo (" + numLinhasRegistos + ")");
      }

      List<SensorRecord> records = new ArrayList<>();
      for (int i = 1; i < lines.length; i++) {
        String linha = lines[i].trim();
        if (!linha.isEmpty()) {
          records.add(SensorRecord.fromLine(linha, nParticoes));
        }
```

```
28        }
29        return new Batch(address, port, inc, hb, lastSeq, records);
30    }
31
```

Explicação detalhada: Este método é chamado pelas réplicas, produtor e processador quando recebem uma mensagem do IPFS. Começa por separar o cabeçalho das linhas de registo (com `msg.split("\n")`). Valida se o cabeçalho tem exatamente 7 campos e se o primeiro campo é `"LOTE"`. Se não for, lança exceção (isto protege contra mensagens corrompidas ou de outro tipo). Extrai os valores do cabeçalho e faz uma **validação crucial**: compara o `nHeader` (o número declarado no cabeçalho) com o número de linhas de registo efetivamente recebidas (`lines.length - 1`). Se não coincidirem, significa que a mensagem foi truncada na rede e o lote é rejeitado por inteiro. Isto garante a **atomicidade**: ou se aplica o lote completo ou não se aplica nada. Por fim, cria uma lista de `SensorRecord` e devolve o objeto `Batch`.

3.3 O Detetor de Falhas (`HeartbeatMonitor.java`)

3.3.1 Método `onHeartbeat()`

```
1     public synchronized void onHeartbeat(long inc, long hb) {
2       if (inc < this.inc) return;                    // incarnacao
    antiga
3       if (inc == this.inc && hb <= this.hb) return;  // duplicado ou
    atrasado
4
5       if (this.inc != -1 && inc != this.inc) {
6         log.accept("REINICIO;p" + partition + ";inc=" + this.inc + "->"
    + inc);
7       } else if (inc == this.inc && hb != this.hb + 1) {
8         log.accept("HB_PERDIDO;p" + partition + ";esperado=" + (this.hb
    + 1) + ";recebido=" + hb);
9       }
10      if (state == State.SUSPEITO) {
11        log.accept("RETOMA;p" + partition + ";inc=" + inc + ";hb=" + hb)
    ;
12      }
13
14      this.inc = inc;
15      this.hb = hb;
16      this.lastNanos = System.nanoTime();
17      this.state = State.ATIVO;
18    }
19
```

Explicação detalhada: Este método é chamado pela thread que recebe os lotes do IPFS. Os primeiros dois `if` são filtros:

- `if (inc < this.inc) return;`: se o lote vier de uma incarnação antiga (mensagem atrasada na rede), é ignorado para não fazer o monitor regredir no tempo.
- `if (inc == this.inc && hb <= this.hb) return;`: se for um heartbeat duplicado ou atrasado dentro da mesma incarnação, também é ignorado.

Depois, se o `inc` for diferente do anterior, regista `REINICIO` (o líder reiniciou). Se o `inc` for o mesmo mas o `hb` saltou um número, regista `HB_PERDIDO`. Se o estado era `SUSPEITO`,

regista `RETOMA` (o líder voltou). Finalmente, atualiza as variáveis `inc`, `hb`, `lastNanos` (com `System.nanoTime()`) e passa o estado para `ATIVO`. O método é `synchronized` porque é acedido pela thread do IPFS e pela thread de verificação (`check`).

3.3.2 Método `check()`

```
    public synchronized void check() {
      long elapsed = System.nanoTime() - lastNanos;
      if (state != State.SUSPEITO && elapsed > timeoutNanos) {
        state = State.SUSPEITO;
        log.accept("SUSPEITA;p" + partition + ";silencio_ms=" + elapsed
/ 1_000_000);
      }
    }

```

Explicação detalhada: Este método é chamado a cada segundo pela thread de verificação (`ScheduledExecutorService`). Calcula o tempo decorrido desde a última receção (`elapsed`). Se este tempo exceder o `timeoutNanos` (15000 ms) e o estado ainda não for `SUSPEITO`, muda o estado e regista o evento com o tempo de silêncio em milissegundos.

3.4 Recuperação Assíncrona na Réplica (`PartitionReplica.java`)

3.4.1 O gatilho no método `receber()`

```
    synchronized void receber(String msg) {
      Batch batch;
      try {
        batch = Batch.decode(msg, n);
      } catch (IllegalArgumentException e) {
        System.out.println("LOTE REJEITADO: " + msg + " | " + e.
getMessage());
        return;
      }

      monitor.onHeartbeat(batch.inc, batch.hb);

      String seqs = batch.records.isEmpty() ? "vazio" :
      batch.records.get(0).seq + ".." + batch.records.get(batch.records.
size() - 1).seq;
      registarMonitor("RECEBIDO;p" + k + ";inc=" + batch.inc + ",hb=" +
batch.hb +
      ",ultimoSeq=" + batch.lastSeq + ",n=" + batch.records.size() + ",
seqs=" + seqs);

      marcarAtividade();
      hostLider = batch.address;
      portoLider = batch.port;

      for (SensorRecord r : batch.records) {
        // ... (logica de simulacao de perda)
        aplicar(r);
      }

      if ((ultimo < batch.lastSeq || retidos.size() > Wire.
LIMITE_RETIDOS) && !emRecuperacao) {
```

```
27          long de = ultimo + 1;
28          long ate = batch.lastSeq;
29          if (!retidos.isEmpty() && retidos.firstKey() > de) {
30            ate = Math.min(ate, retidos.firstKey() - 1);
31          }
32          if (de <= ate) {
33            emRecuperacao = true;
34            registarMonitor("RECUPERACAO;p" + k + ";de=" + de + ";ate=" +
    ate);
35            final long fDe = de;
36            final long fAte = ate;
37            new Thread(() -> executarRecuperacao(batch.address, batch.port
    , fDe, fAte)).start();
38          }
39        }
40      }
41
```

Explicação detalhada: O método `receber` é chamado quando chega um lote do IPFS. Primeiro, descodifica o lote. Se o lote for inválido (truncado), descarta-o e retorna. Depois, chama `onHeartbeat` para atualizar o monitor. Regista o evento `RECEBIDO` no ficheiro de log. Guarda o endereço e porto do líder (para futuras recuperações). Aplica os registos com o método `aplicar` (que verifica duplicados e lacunas). **A parte crucial:** depois de aplicar os registos, verifica se `ultimo < batch.lastSeq`. Isto é, se o último `seq` aplicado é inferior ao `ultimoSeq` do lote, há uma lacuna. Se houver lacuna e a flag `emRecuperacao` estiver a `false`, ativa a flag, regista o evento `RECUPERACAO` e lança uma nova thread para executar a recuperação TCP.

3.4.2 O método `executarRecuperacao()`

```
1     void executarRecuperacao(String targetHost, int targetPort, long de,
      long ate) {
2       List<SensorRecord> temporarios = new ArrayList<>();
3       try (Socket s = new Socket()) {
4         s.connect(new InetSocketAddress(targetHost, targetPort), Wire.
    T_TCP_MS);
5         s.setSoTimeout(Wire.T_TCP_MS);
6
7         BufferedReader in = new BufferedReader(new InputStreamReader(s.
    getInputStream(), StandardCharsets.UTF_8));
8         PrintWriter out = new PrintWriter(new OutputStreamWriter(s.
    getOutputStream(), StandardCharsets.UTF_8), true);
9
10        out.println("PEDIDO " + de + " " + ate);
11
12        boolean fim = false;
13        String linha;
14        while ((linha = in.readLine()) != null) {
15          if (linha.equals("FIM")) { fim = true; break; }
16          if (linha.startsWith("ERRO")) { throw new IOException("lider
    respondeu " + linha); }
17          temporarios.add(SensorRecord.fromLine(linha, n));
18        }
19        if (!fim) throw new IOException("resposta interrompida antes de
    FIM");
20
```

```
        synchronized (this) {
          for (SensorRecord r : temporarios) {
            retidos.put(r.getSeq(), r);
            recuperados++;
          }
          escreverRetidosEmSequencia();
        }
      } catch (IOException | IllegalArgumentException e) {
        ligacoesFalhadas++;
        System.out.println("RECUPERACAO FALHOU: " + e.getMessage());
      } finally {
        synchronized (this) { emRecuperacao = false; }
      }
    }

```

Explicação detalhada: Este método corre numa thread separada. Abre uma ligação TCP com o líder e envia o pedido `PEDIDO de ate`. Lê os registos do socket e guarda-os numa lista `temporarios`. **Não os aplica logo**, porque se a ligação falhar a meio, a réplica ficaria com dados parciais. Só quando recebe a linha `FIM` é que entra no bloco `synchronized(this)` (para garantir que a thread do IPFS não está a mexer nos `retidos` ao mesmo tempo) e aplica os registos recuperados. No bloco `finally`, repõe a flag `emRecuperacao` a `false`, mesmo que tenha havido erro.

3.5 Relógios Lógicos no Processador (`AlarmProcessor.java`)

```
1     private void receber(int k, String msg) {
2       Batch batch;
3       try {
4         batch = Batch.decode(msg, n);
5       } catch (IllegalArgumentException e) {
6         System.out.println("REJEITADO no processador: " + e.getMessage()
    );
7         return;
8       }
9
10      monitores[k].onHeartbeat(batch.inc, batch.hb);
11
12      for (SensorRecord r : batch.records) {
13        if (r.getSensorId().startsWith("ALARM-")) continue;
14        if (!seenSeqs[k].add(r.getSeq())) continue;
15
16        String alarmMsg = null;
17        int targetPartition = -1;
18
19        synchronized (lock) {
20          lamport = Math.max(lamport, r.lamport);
21          vector.merge(r.vector);
22
23          if (r.getTemperature() >= Wire.LIMIAR_ALARME) {
24            String alarmSensor = "ALARM-" + r.getSensorId();
25            targetPartition = part.of(alarmSensor);
26            alarmMsg = Wire.in(alarmSensor, r.getTemperature(), lamport,
    vector.copy());
27            System.out.println("ALARME: " + r.getSensorId() + " p" + k +
    "#" + r.getSeq() + " -> p" + targetPartition);
```

```
28            }
29          }
30
31          if (alarmMsg != null) {
32            if (monitores[targetPartition].state() == HeartbeatMonitor.
    State.SUSPEITO) {
33              registarMonitor("AVISO: lider p" + targetPartition + "
    suspeito; alarme " + r.getSensorId() + " pode perder-se");
34            }
35            try {
36              pubsub.publish(Wire.topicIn(targetPartition), alarmMsg);
37            } catch (IOException | InterruptedException e) {
38              System.out.println("Erro ao publicar alarme: " + e.
    getMessage());
39            }
40          }
41        }
42      }
43
```

Explicação detalhada: O processador recebe o lote, descodifica e atualiza o monitor de heartbeat. Depois itera sobre os registos do lote (fazendo skip aos que começam por `ALARM-` e aos duplicados). **O bloco synchronized(lock) é crucial:** o `lamport` e o `vector` são variáveis globais partilhadas por todas as threads (p0, p1, p2). Se não houvesse lock, duas threads podiam tentar atualizar o vetor ao mesmo tempo, corrompendo os dados. O lock garante que apenas uma thread de cada vez faz o `merge`. Se a temperatura for superior ao limiar, cria uma mensagem de alarme com os relógios lógicos atuais e calcula a partição de destino. A publicação é feita **fora do lock** porque é uma operação de rede lenta; se estivesse dentro, bloqueava as outras threads. Se o líder de destino estiver `SUSPEITO`, regista um aviso mas publica na mesma (porque o alarme pode perder-se, mas o processador não pode reter).

4 Perguntas Fundamentais do Agente e Respostas

1. **P: Porque é que o líder tem de ler o `ultimoSeq` dentro da secção crítica?**
   R: Para garantir a consistência entre o cabeçalho e os dados. Se a leitura fosse feita fora, uma thread podia adicionar um `IN` à lista depois do `ultimoSeq` ter sido lido, fazendo com que o cabeçalho indicasse um `ultimoSeq` maior do que os registos efetivamente transportados. Isso geraria falsas lacunas.

2. **P: Porque é que a réplica não pode simplesmente descartar um `LOTE` com `inc` antigo?**
   R: O `HeartbeatMonitor` ignora o sinal de vida de uma incarnação antiga (para não regredir no tempo), mas os registos que vêm nesse lote (se forem novos) devem ser aplicados. A validação do sinal de vida é independente da aplicação de dados.

3. **P: Como distingues uma falha real de uma falsa suspeita no `HeartbeatCheck`?**
   R: Cruzando o evento `SUSPEITA` com o evento `RETOMA` seguinte. Se houver um `RETOMA` com o mesmo `inc`, foi falsa suspeita (o líder apenas teve uma pausa). Se houver mudança de `inc` ou o ficheiro terminar, é falha real.

5 Simulado de Defesa (Perguntas e Respostas)

Pergunta 1: O que protege o `synchronized (LOCK)` no líder e porque a publicação é fora?

Resposta: Protege o *snapshot* atómico (troca da lista `pendentes` e leitura do `ultimoSeq`). A publicação é fora porque a rede é lenta; se estivesse dentro, bloqueava a thread de receção de novos `IN`s, degradando a performance.

Pergunta 2: A réplica fica bloqueada à espera da recuperação TCP?

Resposta: Não. A recuperação corre numa thread separada. A thread principal continua a ouvir o IPFS e a atualizar o monitor. A flag `emRecuperacao` evita pedidos duplicados.

Pergunta 3: Porque o detetor diz `SUSPEITO` e não `MORTO`?

Resposta: Num sistema assíncrono, um líder pode estar apenas lento (ex: Garbage Collector) e não morto. Dizer `MORTO` implicaria *failover* automático (eleição de líder), o que está fora do âmbito. O `SUSPEITO` apenas alerta que a rede ou o líder estão instáveis.

Pergunta 4: Porque os heartbeats não incrementam `L` e `V`?

Resposta: Os relógios lógicos só avançam com eventos de dados (leituras). Se um heartbeat vazio incrementasse o `V[k]`, quebraria o invariante `V[k] = seq`, porque o vetor estaria à frente do número de registos reais.

Pergunta 5: O que acontece às leituras enviadas para um líder suspeito?

Resposta: O produtor avisa que podem perder-se, mas envia-as na mesma. Se o líder estiver mesmo morto, perdem-se para sempre, porque nunca lhes foi atribuído um `seq`. Para não as perder, seria necessário *buffering* no produtor com confirmações (ACK), o que exigiria consenso (fora do âmbito).

Pergunta 6: `synchronized(this)` na réplica e `synchronized(lock)` no processador.

Resposta: Na réplica, `this` tranca a partição específica, garantindo que a recuperação TCP e a receção do IPFS não colidem. No processador, o `lock` explícito tranca o vetor global (partilhado por todas as partições), garantindo que as threads de p0, p1 e p2 não corrompem o vetor ao mesmo tempo.

Pergunta 7: O que é o `inc` e porque ignoramos os antigos?

Resposta: O `inc` é a incarnação (timestamp de arranque do líder). Ignoramos os antigos (se `inc < this.inc`) para evitar que mensagens atrasadas na rede (de um líder antigo) façam o monitor regredir no tempo e gerar falsos eventos.

6 Cenários de Falha (Perguntas de Defesa)

Cenário A: O líder falha e reinicia

Resposta: O líder antigo não emite heartbeats. Ao fim de 15s, as réplicas marcam `SUSPEITA`. Quando o novo líder arranca com um novo `inc`, envia um lote vazio com `ultimoSeq` atualizado. A réplica deteta que tem um `seq` inferior e aciona a recuperação TCP. O campo `ultimoSeq` no lote vazio revela a lacuna mesmo não havendo mais leituras.

Cenário B: Falsa suspeita (Pausa do líder com `kill -STOP`)

Resposta: O líder fica suspenso durante 20s. As réplicas geram `SUSPEITA`. Quando o líder retoma (`kill -CONT`), envia um heartbeat. A réplica gera `RETOMA`. O `HeartbeatCheck` classifica-a como falsa suspeita porque o `inc` do `RETOMA` é igual ao da `SUSPEITA`.

Cenário C: A rede falha no meio da recuperação TCP

Resposta: A réplica lança a recuperação, mas a ligação cai antes do `FIM`. Os dados são guardados numa lista `temporarios` e descartados. A flag `emRecuperacao` volta a `false` no `finally`. A consistência é mantida porque nada é aplicado pela metade; a réplica tentará novamente no próximo heartbeat que revelar a lacuna.

Cenário D: O lote chega corrompido (truncado)

Resposta: O `Batch.decode` lança exceção, porque o `n` do cabeçalho não corresponde ao número de linhas. O lote é rejeitado por inteiro. Nenhum registo parcial é aplicado.